

Methodology and Iteration Log

Project: Financial Complaints Analyzer — AI Developer Technical Assessment **Submitted to:** CocaColaHBC

1. Brief

The assessment had two deliverables.

Task 1 — analysis. Explore a consumer-complaint dataset, identify the complaint topics using an unsupervised approach, build a usable taxonomy, and (bonus) train a classifier on that taxonomy.

Task 2 — agentic layer. Build an AI agent on top of the classifier and the complaint history that helps an operations team triage, route, and act on incoming complaints.

The assessment text was explicit that the evaluators care about reasoning, strategy, ownership of the full process, and practical decisions under ambiguity — not the highest metric.

2. Dataset

Property	Value
File	complaints_flat.csv
Rows	78,313
Columns	18
Rows with narrative text	21,072 (26.9 %)
After preprocessing (min 50 chars)	20,998
Time span	March 2015 – early 2021
Source	CFPB public consumer complaint extract; visibly Chase-heavy

The narrative field is gated by consumer consent, so 73 % of rows have no complaint text. All downstream modelling runs on the 21 k narrative subset.

Verified data-quality findings

- company_public_response is essentially 100 % null — dropped early.
- consumer_disputed (~46 % null) and tags (~86 % null) both contain real signal — kept. Tags encode Servicemember / Older American — used later by the agent's urgency rules.
- xxxx redaction tokens carry semantic signal in n-gram context ("social security xxxx", "xxxx street"). Kept everywhere except as bare unigrams.
- The raw file mixes old and new CFPB product labels (17 unique). Merged to 12 canonical products in Section 2.

5. `timely` is Yes for ~99 % of rows — zero variance for classification. Dropped from classifier features, kept in agent state for the rare No case (urgency elevator).
6. Mortgage dominates the full file (~29 %) but is only third (16 %) in the narrative subset. Class imbalance is real but milder than expected.
7. Date trend is stationary at 200 – 350 complaints / month from 2015 to early 2020, then surges to ~450 / month mid-2020 (COVID-19 effect). End-of-series drop is a data-truncation artefact, not a real decline.

3. Topic Modelling — Task 1, Sections 4 – 6

Why BERTopic

We chose BERTopic over LDA / NMF because consumer-complaint text is noisy and semantically diverse. *"I never authorised this charge"* and *"unauthorized transaction"* share no keywords but mean the same thing — sentence embeddings handle this, bag-of-words cannot. BERTopic packages the right pipeline (embed → UMAP → HDBSCAN → c-TF-IDF) into one cohesive workflow, and the c-TF-IDF representations give us free, readable topic keywords without a separate naming step.

Embedding model

`sentence-transformers/all-MiniLM-L6-v2`. 384-dim, CPU-fast, less than 5 % quality loss versus larger models on short text. Embeddings are computed once and cached to `outputs/embeddings.npy`; the same vectors are reused for BERTopic clustering, the Logistic Regression feature stack, and ChromaDB indexing. `normalize_embeddings=True` so cosine similarity is just a dot product downstream.

Hyperparameters and iterations

The first BERTopic fit, with the default-recommended `min_cluster_size=50` and `min_samples=5`, produced **51 topics with 40.6 % noise**. Too many topics, very high noise.

We raised `min_cluster_size` to 100 (forces smaller clusters to dissolve back into noise or absorb into nearest neighbours) and lowered `min_samples` to 3 (lets borderline points join clusters more readily). The combined effect was **25 topics with 40.7 % noise**.

The noise rate barely moved. That is by design — HDBSCAN refuses to force outliers into clusters. So we added a **post-hoc outlier-reduction step** (new cell 19): `topic_model.reduce_outliers(..., strategy="embeddings", embeddings=embeddings)` followed by `topic_model.update_topics(...)`. This reassigns each noise document to its nearest topic centroid using the cached embeddings, and rebuilds the c-TF-IDF representations.

After reduction: **25 topics, 0 % noise**, every document carries a usable label for classifier training. The catch-all (Topic 0) absorbed most of the previously-noise documents and ended up at ~42 % of the dataset — noted as a known limitation, not a bug.

c-TF-IDF vectorizer — the bigram fix

The first run's topic keywords were dominated by English stopwords (`the`, `my`, `was`, `that`, `and`) and the bare `xxxx` token. The default `ngram_range=(1, 2)` with `stop_words='english'` was supposed to filter stopwords, but BERTopic's safetensors save/load roundtrip drops the vectorizer, so when `update_topics` ran after `reduce_outliers`, it used a default vectorizer with no stopwords.

We rewrote the vectorizer to `ngram_range=(2, 3)` with `stop_words=ENGLISH_STOP_WORDS + ['xx']` and pinned it as the explicit `vectorizer_model` for the post-reduction `update_topics` call. The new representations surface contextual phrases:

- `social security xxxx, power attorney, account xxxx, xxxx street` — what got redacted
- `escrow account, property taxes, loan modification, short sale` — operational themes
- `chapter 7, identity theft, fair credit reporting act` — high-urgency vocabulary

Bigrams dropped some clean single-word labels like `escrow` or `overdraft` alone, but bigrams are usually more descriptive anyway. Trade accepted.

Taxonomy — the 25 labels

After inspecting keywords + topic × product cross-tabs + sample texts per topic, we mapped each topic id to a human-readable label. Three of the labels initially had Chase / JPMorgan in them (the source distinctive vocabulary), and we generalised those so the deliverable is bank-agnostic:

Old label (during development)	Final generic label
JPMorgan Chase Mortgage Disputes (Formal)	Formal/Legal Mortgage Disputes
Chase Amazon Card Issues	Co-Branded Retail Card Issues
JPMCB Credit Report Hard Inquiries	Bank Credit Report Hard Inquiries

The other 22 labels (overdraft fees, mortgage escrow, identity theft / FCRA, senior / POA account access, etc.) were already generic.

Topic 0 ("General Account & Customer Service Issues") is the ~42 % catch-all and is treated as the low-information default. The classifier should learn to under-predict it on specific complaints and reach for the more specific labels first.

4. Classifier — Task 1, Sections 7 – 10

We trained two classifiers on the BERTopic-labelled documents (pseudo-label self-training) and compared them.

Logistic Regression (baseline)

Feature stack:

- **TF-IDF** over the full text — 10,000 bigram features, sublinear-TF, `min_df=5`.
- **Sentence embeddings** (the same 384-dim vectors used by BERTopic and ChromaDB) — captures paraphrase, fixes the bag-of-words blind spot.
- **Metadata one-hots** — `product`, `submitted_via`. `Not issue / sub_issue` (too correlated with topic labels, would leak). `Not timely` (zero variance per the EDA finding).

Training: stratified 70/10/20 train/val/test split, `class_weight='balanced'`, `solver=saga`, `C=1.0`, `max_iter=1000`.

Result: macro-F1 **0.7915**, accuracy **0.8012**, 40 % of test-set predictions below the 0.60 confidence threshold
→ HITL queue.

DistilBERT + LoRA (PEFT)

Base: distilbert-base-uncased. **LoRA:** rank 8, alpha 16, dropout 0.1, target modules q_lin and v_lin. Approximately 1.2 M trainable parameters out of 67 M (1.8 %). Trained at max_length=256 tokens, batch_size=16, fp16=True, 4 epochs with early-stopping patience 2, custom WeightedTrainer to apply class weights.

Result: macro-F1 **0.7067**, accuracy **0.7212**, 25 % HITL rate.

Why LogReg won by ~11 points

1. **More information per example.** LogReg sees TF-IDF over the full text *plus* embeddings over the full text *plus* metadata one-hots. LoRA only sees text capped at 256 tokens (median complaint is ~250 tokens, so half the corpus is truncated for LoRA), and gets no metadata signal.
2. **CFPB complaints are vocabulary-driven.** Topics carry distinctive keyword signatures — escrow, overdraft, inquiry, lien, short sale, refinance. TF-IDF + class-weighted LogReg captures these directly. A small fine-tuned transformer would only beat that if it had more data, longer context, or the task needed semantic reasoning beyond keywords.
3. **Confidence calibration.** LogReg sends 40 % of predictions to HITL; LoRA only 25 %. LoRA is more confident but less accurate — over-confidence is a worse failure mode for an Ops triage system. The classifier should route to a human precisely when it is unsure.

Constraints of this comparison

- **Feature parity is not feature equality** — LoRA's smaller information set is a real handicap that we did not equalise. A fairer comparison would prepend product=mortgage. channel=web. to the LoRA input, or concatenate the sentence embedding into the classifier head, or both.
- **Text-length truncation** falls disproportionately on the longer mortgage topics — LoRA is penalised hardest on Mortgage Loan Modification, Formal/Legal Mortgage Disputes, Mortgage Refinance / Rate Disputes, Mortgage Short Sale, and Mortgage Payment Processing.
- **Different training regimes** — both apply class-balanced weighting, but LogReg's class_weight='balanced' scales gradients per class; LoRA's WeightedTrainer applies the same weights on a softmax cross-entropy. Different geometry.
- **Identical labels, identical Topic 0 burden** — both models under-predict the ~42 % catch-all, which is the desired failure mode. LoRA gives up more ground on Topic 0 (recall 0.63) than LogReg (recall 0.72) to win on the smaller classes.

Tuning levers if we wanted LoRA to surpass LogReg: max_length=512, concat metadata into the prompt, LoRA rank 16, more epochs, smoothed class weights, lower learning rate. We documented these explicitly rather than chase the metric.

Decision

LogReg ships as the production classifier (CLASSIFIER_TO_USE = 'logreg'). LoRA artefacts and the comparison report stay in the notebook as a Section 10 deliverable.

5. Agentic Layer — Task 2

Framework

LangGraph. Native conditional edges for HITL branching, native `interrupt_before` for human-in-the-loop, and clean integration with Gemini via `langchain-google-genai`. The state machine is visually presentable, which helps in the interview.

State graph

classify → assess_urgency → retrieve_similar → draft_case_note → review_gate

■ human_review

■ auto_output

Node	What it does	Cost
classify	Local Logistic Regression or LoRA prediction → topic label + confidence.	\$0
assess_urgency	Regex over high-risk keywords → instant HIGH. Else vulnerability tag → HIGH. Else Gemini call for the MEDIUM / LOW grey zone.	\$0 in the regex tier, ~50 tokens in the LLM tier.
retrieve_similar	ChromaDB cosine search filtered by matching topic label → 3 historical complaints with their company_response outcomes.	\$0
draft_case_note	Gemini call grounded in classifier output, urgency, and the 3 ChromaDB precedents. 4 sections: Summary, Key Issues, Recommended Action, Risk Flags.	~500 tokens per case note.
review_gate	Decides human_review vs auto_output from three flags.	\$0

Urgency rules — the regex tier

```
HIGH_URGENCY_PATTERNS = [  
    r"\b(attorney|lawyer|lawsuit|sue|court|legal action)\b",  
    r"\b(cfpb|consumer financial protection bureau)\b",  
    r"\bidentity theft\b",  
    r"\bfraud\b",  
    r"\b(foreclosure|eviction|repossession|repo)\b",  
    r"\b(bankruptcy|chapter 7|chapter 11|chapter 13)\b",  
    r"\bsuicid",  
    r"\bhospital\b",  
]
```

Each pattern targets an operational risk: legal escalation, regulator already engaged, fraud, imminent loss of asset, protected legal status, distress, health crisis. The rules fire deterministically — no LLM call, fully auditable. Bankruptcy or fraud keywords short-circuit the Gemini judgment entirely.

Vulnerability tags (`Servicemember`, `Older American`) are a second deterministic HIGH trigger.

Only when both deterministic tiers fail does the agent call Gemini for the MEDIUM / LOW distinction. We chose Gemini 2.5 Flash Lite for its generous free-tier quota; the prompt asks for a JSON object with `urgency` and `reason`.

HITL triggers

Three independent reasons can route a packet to a human:

1. **Confidence < 0.60** — classifier is unsure.
2. **Urgency = HIGH** — regulator, legal, fraud, bankruptcy, distress, or vulnerable customer.
3. **No similar precedents found** — novel complaint type the ops team has not seen before.

The `human_review_reason` is preserved on the state and surfaces in the formatted triage packet.

Vector database

ChromaDB, local and persistent at `outputs/chroma_db/`. Cosine similarity (the embeddings are normalised). Each entry stores the complaint text plus metadata: `topic_label`, `company_response`, `product`, `sub_product`, `timely`, `complaint_id`.

Leakage-free demo split. The index covers Task 1's train + val splits. The demo set is Task 1's `idx_test` (saved by Section 7) — held out from **both** the classifier and ChromaDB. The agent demo therefore tests on documents none of the components have ever seen. We assert `len(set(idx_index) & set(idx_test)) == 0` at index time as a hard check.

Case note grounding

The `draft_case_note` Gemini prompt receives:

- the original complaint text,
- the classifier's topic label and confidence,
- the urgency and its reason,
- the three retrieved similar complaints with their outcomes.

This grounding prevents the LLM from inventing resolutions. When the demo's similar cases show "Closed with explanation" 2 of 3 times and "Closed with monetary relief" 1 of 3 times, the case note typically frames the recommended action against those precedents.

Daily briefing — Section 14

A batch helper that processes a day's worth of incoming complaints using only the local classifier (no per-complaint Gemini call) and then issues **one** Gemini call to synthesise a morning brief for the ops manager. Cost-efficient: a 250-complaint day costs about 500 tokens total.

6. Notable Iteration Log

The pipeline didn't converge in a single shot. The iterations below are the meaningful ones.

#	Change	Why
1	Initial BERTopic fit at default params produced 51 topics, 40.6 % noise.	Out-of-the-box settings were too permissive.
2	Raised <code>min_cluster_size</code> 50 → 100, lowered <code>min_samples</code> 5 → 3, deleted cache, re-fit.	Target 12 – 25 topics with < 20 % noise. Got 25 topics; noise barely changed.
3	Added cell 19 — post-hoc <code>reduce_outliers(strategy="embedding")</code> instead of dropping them from classifier training.	Recovered the 8.5 k noise through use of embeddings instead of dropping them from classifier training. Net effect: 25 topics, 0 % noise.
4	Discovered topic keywords were leaking English stopwords because BERTopic's <code>safetensors</code> save dropped the vectorizer.	The vectorizer wasn't persisted across the save/load roundtrip.
5	Rewrote vectorizer to <code>ngram_range=(2, 3) + ENGLISH_STOP_WORDS + ['xx']</code> . Added cell 20 to refresh representations on the cached cluster assignments.	Bigrams reveal what <code>xxxx</code> redactions stood for (social security <code>xxxx</code> , power attorney). Cleaner labels.
6	Filled <code>TOPIC_LABELS</code> with 25 entries derived from keywords + cross-tabs + samples.	Manual taxonomy step (assessment requirement).
7	Renamed 3 Chase / JPMorgan-specific labels to generic equivalents.	Project was for CocaColaHBC, not Chase. Generic labels keep the work bank-agnostic.
8	Dropped <code>timely</code> from classifier features.	EDA showed ~99 % "Yes" — zero variance.
9	Trained Logistic Regression (~30 s) and DistilBERT-LoRA (~10 min on RTX 2060).	Comparison is the assessment's bonus deliverable.
10	Comparison: LogReg 0.79 / LoRA 0.71 macro-F1. Shipped LogReg.	Simpler model, fewer features, 20× faster training, better-calibrated confidence.

11	Split the work into two notebooks: <code>complaint_analysis.ipynb</code> (Task 1) and <code>complaint_agent.ipynb</code> (Task 2).	Task 2 only needs outputs/ from Task 1; splitting makes each notebook independently runnable and reviewable.
12	Task 2: changed the 90 / 10 demo split to use Task 1's <code>idx_test</code> directly.	Prevents leakage between ChromaDB and the classifier training set.
13	Built routing map for all 25 topics → operational teams.	Default fallback was firing on every packet because the original placeholder keys didn't match the real labels.
14	Strengthened <code>assess_urgency</code> parser to handle Gemini returning <code>resp.content</code> as either a string or a list of message parts.	Newer Gemini models return parts. The string-only assumption was failing all urgency LLM calls with <code>'list'</code> object has no attribute <code>'strip'</code> .
15	Removed duplicate routing prints — moved all output to the formatter.	The agent nodes were calling <code>print()</code> as a side effect while the formatter also printed, so each packet's decision showed up twice.
16	Switched LLM from <code>gemini-2.0-flash</code> to <code>gemini-2.5-flash-lite</code> .	Free-tier quota issue: the older model returned <code>RESOURCE_EXHAUSTED limit: 0</code> on the user's key.

7. Final Results

Task 1

- 25 BERTopic topics, 0 % noise after post-hoc reduction.
- 20,998 labelled documents.
- Logistic Regression: macro-F1 **0.7915**, accuracy **0.8012**, 40 % HITL.
- DistilBERT + LoRA: macro-F1 **0.7067**, accuracy **0.7212**, 25 % HITL.
- LogReg shipped as the production classifier.

Task 2

- ChromaDB index: 16,798 historical complaints (train + val), held out from the demo set.
- Agent state machine with 7 nodes and 3 independent HITL triggers.
- Regex urgency rules fire on legal / fraud / bankruptcy / FCRA / vulnerability without an LLM call.

- Gemini handles the MEDIUM / LOW grey zone and the case-note drafting.
- Sample run on 5 held-out complaints routed:
- 2 auto (Account Opening Bonus → Retail Banking; Auto Loan Title → Auto Lending)
- 3 to human review (FCRA willful non-compliance, bankruptcy, low classifier confidence)
- Daily-briefing function processes a full day's worth of complaints in one Gemini call.

8. Known Limitations

- **Topic 0 is ~42 % of the data.** BERTopic's auto-reduction + the post-hoc outlier redistribution combined to dump miscellaneous complaints into the broadest cluster. The classifier learns it as a low-information catch-all. Possible fix: drop Topic 0 from classifier training and let those complaints fall to low-confidence → HITL.
- **Dataset is Chase-only.** The model generalises in principle — features are vocabulary-driven, labels are generic — but a multi-bank retrain would be needed before deployment.
- **Stratified random split, not time-ordered.** Right for assessment; for production, a time-ordered split would be required to detect concept drift (e.g. pre-COVID training, post-COVID inference).
- **Gemini free-tier quota** caps the agent demo at ~20 LLM calls per day (`gemini-2.5-flash-lite`). The agent degrades gracefully when the LLM is unavailable: regex urgency still routes correctly, case notes show a placeholder.

9. Future Work

- **Time-ordered evaluation.** Train on Jan 2015 – Dec 2019, test on Jan 2020 – Apr 2021. Measure drift.
- **Drop Topic 0 from classifier training.** Cleaner labels, smaller dataset, ambiguous complaints fall to HITL by design.
- **Enrich LoRA features.** `max_length=512`, prepend product/channel into the prompt, concat sentence embedding into the classifier head, retrain. Expected gain: 3–5 macro-F1 points.
- **DPO loop.** Ops agents accept / edit / reject case notes → preference pairs → DPO fine-tuning of the case-note generator. Aligns the LLM with actual ops team taste over time.
- **Production case-note prompts.** Add bank-specific compliance disclaimers, redact PII before sending to the LLM, log prompt + response pairs for audit.
- **Char n-grams in LogReg.** Catches OCR / typo variants. Likely 1–2 macro-F1 points.